INSTITUTO FEDERAL
Ceará
Campus Tianguá

TRABALHO COMPUTACIONAL 02 – ANÁLISE LÉXICA, ANÁLISE SINTÁTICA E COMPILADOR BÁSICO	
CURSO	Bacharelado em Ciência da Computação
DISCIPLINA	Compiladores
PROFESSOR	Adonias Caetano de Oliveira
EQUIPE	NO MÁXIMO 6 PESSOAS

Finalidade

O trabalho computacional tem como objetivo aplicar, de forma prática, os principais conceitos relacionados às linguagens formais e ao processo de compilação, integrando a modelagem e simulação de Linguagens Regulares e Linguagens Livres de Contexto com o desenvolvimento de componentes básicos de compiladores. Para isso, serão utilizadas as ferramentas JFLAP e GALS na construção, simulação e análise de autômatos e gramáticas; o Verto para observar e compreender as etapas do processo de compilação; e as linguagens C, Java e Python para implementação de analisadores léxicos. Por fim, será desenvolvido um compilador básico em Python, utilizando PLY ou RPLY, permitindo integrar conceitos de análise léxica e sintática e consolidar a compreensão prática das principais etapas envolvidas na construção de compiladores.

ORGANIZAÇÃO EM EQUIPES

A turma deve se dividir em equipes e escolher o conjunto de questões referentes a numeração de equipe. Somente as questões gerais A e B serão resolvidas e apresentadas por todas as equipes. Para demais questões distribuídas por equipe não deve ocorrer repetição de apresentações de um mesmo conjunto de questões.

Por isso é necessário que cada equipe indique a numeração de equipe, bem como, cite os nomes de todos os membros da equipe como comentário do mural da atividade no classroom. A prioridade de escolha da numeração de equipe será baseada na data e horário do comentário da atividade classroom.

A apresentação será remota assíncrona com entrega para o dia marcado pelo professor segundo o cronograma da disciplina com prazo de entrega dos slides (optativo) e códigos JFLAP, GALS, Verto, C, Java e Python pelo **link GitHub** no classroom até 18/12/2026. **É necessário** que todos os membros da equipe apresentem sua parte. Cada equipe terá no **máximo 45 de minutos** de apresentação.

A equipe deve treinar a apresentação e evite ensinar passos que fogem do escopo da apresentação como instalação e configuração de ambiente.

Instruções para Entrega do Trabalho em Vídeo

O trabalho deverá ser entregue **exclusivamente em formato de apresentação em vídeo**, seguindo rigorosamente as orientações abaixo:

1. Formato da Entrega

A apresentação deverá ser disponibilizada por meio de **um link público ou não listado**, utilizando uma das seguintes plataformas:

- **Google Drive:** compartilhar o vídeo com a permissão de acesso "Qualquer pessoa com o link pode visualizar".
- **OneDrive:** compartilhar o vídeo por meio de um link que permita sua visualização sem necessidade de solicitar acesso.
- **YouTube:** disponibilizar o vídeo como **público** ou **não listado**. Vídeos configurados como privados não serão aceitos.

O link deverá estar funcionando no momento da entrega e permanecer acessível durante o período de avaliação. Vídeos com acesso bloqueado, links incorretos ou que exijam autorização adicional serão considerados **não entregues**.

2. Conteúdo da Apresentação

O vídeo deverá apresentar, de maneira clara, organizada e objetiva, as soluções das questões propostas no trabalho, observando as seguintes orientações:

- Cada discente deverá identificar-se, informando seu nome antes de iniciar sua parte da apresentação.
- A apresentação deverá concentrar-se na **explicação, fundamentação e justificativa das soluções previamente resolvidas**, demonstrando a compreensão dos conceitos teóricos e dos procedimentos utilizados.
- **Não é necessário resolver as questões do zero durante a gravação.** Os discentes deverão apresentar as soluções já elaboradas, explicando o raciocínio adotado, as etapas relevantes, os conceitos envolvidos e os motivos que justificam as respostas obtidas.
- Cada discente deverá **utilizar obrigatoriamente sua própria voz** na apresentação, sendo **proibido o uso de ferramentas de IA para gerar, sintetizar, clonar ou substituir sua fala**, inclusive em narrações de vídeos previamente gravados. A utilização de voz artificial em substituição à voz do apresentador não será aceita.
- Os discentes poderão utilizar slides, imagens, esquemas, anotações digitais ou outros recursos visuais para apresentar as soluções, desde que esses materiais estejam legíveis e contribuam para a compreensão das explicações. Entretanto, **esses recursos não deverão substituir a apresentação em slides quando sua utilização for expressamente exigida nas orientações do trabalho.**
- A avaliação considerará não apenas a correção das respostas, mas também a **capacidade de explicar, conceituar e justificar as soluções apresentadas**, evidenciando o domínio dos conteúdos estudados.

3. Qualidade Técnica

- **Áudio:** a voz do apresentador deverá estar clara, audível e compreensível durante toda a exposição.
- **Imagem:** o vídeo deverá apresentar qualidade suficiente para permitir a leitura de slides, questões, expressões, diagramas e demais materiais utilizados.
- **Organização:** a apresentação deverá seguir uma sequência lógica, facilitando o acompanhamento das explicações.
- **Integridade:** o vídeo deverá estar completo, sem cortes, interrupções ou problemas técnicos que comprometam a compreensão do conteúdo.
- **Narração:** a apresentação deverá ser realizada com a voz natural do próprio discente, sem substituição por sistemas de síntese de voz ou narração gerada por IA.

Importante: O objetivo da atividade é avaliar a compreensão dos conteúdos e a capacidade dos discentes de comunicar e justificar as soluções desenvolvidas. Portanto, a apresentação não deverá limitar-se à leitura das respostas nem à reprodução mecânica de procedimentos, mas demonstrar o entendimento dos conceitos e fundamentos utilizados na resolução das questões.

CRITÉRIOS DE AVALIAÇÃO:

A Tabela 2 apresenta os critérios de avaliação:

Tabela 2 – Critérios de avaliação da apresentação

AVALIAÇÃO DA EQUIPE			
CRITÉRIOS		NOTA	OBSERVAÇÕES
NOTA	**Organização da equipe**	0 até 10	Diz respeito se há um roteiro lógico de apresentação das soluções, se foi bem dividido a parte de cada um, tanto na apresentação como sobre a divisão de tarefas de revisão do tema e elaboração das soluções e explicações.
	Corretude das soluções	0 até 10	Se as soluções apresentadas de fato resolvem corretamente as questões OU a explicação é correta e completa.
	Eficiência das soluções	0 até 10	Se não há estados ou transições inúteis, tais como deadlocks ou livelocks. Em relação aos códigos, se eles são eficientes

			no gerenciamento de memória, estruturas de dados e tempo de execução.
	Tempo de apresentação (Máx. 45 min)	0 até 10	Se o tempo máximo foi respeitado
	Avaliação oral (Nota individual)	0 até 10	Se cada discente domina bem sua parte de apresentação e na explicação de conceitos relacionados e na justificativa das decisões de projeto demonstrando confiança e tranquilidade, conforme a faixa de notas da Tabela 3
	NOTA FINAL	Média das Notas	A nota final individual poderá ser descontada conforme os critérios da Tabela 4

Tabela 3 – Faixa de nota da qualidade da apresentação oral individual

Avaliação Qualitativa	Faixa de Nota (0 a 10)
Péssima	0,0 – 1,0
Péssima para Ruim	1,1 – 2,5
Ruim	2,6 – 4,0
Ruim para Regular	4,1 – 5,0
Regular	5,1 – 6,0
Regular para Boa	6,1 – 7,0
Boa	7,1 – 8,5
Boa para Ótima	8,6 – 9,5
Ótima	9,6 – 10,0
Excelente	12,0

Tabela 4 – Critérios de desconto de pontos sobre a nota final devido a comportamentos inadequados durante ou após a apresentação

AVALIAÇÃO DA EQUIPE

CRITÉRIOS	PERCENTUAL DE PONTOS DESCONTADOS	OBSERVAÇÕES
Falta de respeito	50% a 75%	Interromper o docente ou aluno de forma grosseira ou desdenhosa com as perguntas ou comentários. Demonstrar impaciência ou desinteresse durante os questionamentos do docente. Postar nas redes sociais ou outros meios qualquer comentário ou foto ou vídeo com o objetivo de humilhar ou debochar de outras equipes, ou sobre o processo de avaliação ou o docente. Deve ocorrer uma boa relação entre os colegas da equipe durante a apresentação como fazer interrupções educadas ou esperar a vez da fala ou citar respeitosamente a fala do colega.
Despreparo	25% a 50%	Não conhecer bem o tema de apresentação ou seminário, não ter domínio sobre o conteúdo da apresentação, não conseguir responder às perguntas de forma clara e objetiva. Ficar excessivamente apenas lendo slides ou folha de consulta ou algum dispositivo durante a apresentação de modo a demonstrar falta de conhecimento.
Linguagem inadequada	15%	Utilizar gírias, expressões coloquiais ou termos técnicos sem a devida explicação, falar de forma muito rápida ou muito baixa, não modular a voz.
Falta de organização	15%	Não ter os materiais de apoio organizados, não saber o tempo de apresentação, não seguir a estrutura da apresentação.
Postura corporal inadequada	5%	Manter-se com os braços cruzados, olhar para baixo, não fazer contato visual com a banca, demonstrar nervosismo excessivo.

Desconhecimento das normas	5%	Não seguir as normas deste documento para a apresentação, como o tempo de apresentação, o formato dos slides, a vestimenta adequada.

Instruções gerais e casos omissos

I. **O docente tem autonomia para definir as atividades avaliativas.**

II. O não cumprimento sobre a quantidade máxima de alunos por equipe resulta em -5 pontos no critério de organização;

III. Este trabalho consiste na entrega dos slides de apresentação (optativo), link de acesso ao vídeo e dos códigos desenvolvidos por meio do classroom;

IV. Basta um membro da equipe entregar efetuar a entrega no classroom;

V. Após a correção desta atividade, em caso de discordância da nota E negativa do professor em atender suas solicitações, é preciso seguir as orientações do artigo 96 do ROD do IFCE.

VI. Não compartilhe arquivos executáveis;

VII. Não será aceito nenhuma resposta das questões anotadas em folhas de caderno ou A4 ou impressa ou digitalizadas ou outro tipo não autorizado nestas instruções;

VIII. Este trabalho inclui como um dos critérios de nota (não o único) a **AVALIAÇÃO ORAL** a fim de medir o nível de conhecimento, confiança e tranquilidade do estudante, em que cada discente deve explicar de modo sucinto as soluções de cada questão;

IX. Se uma equipe (todos os membros) não apresentar nos dias e horários agendados e disponíveis para apresentação oral sem uma justificativa razoável ou mesmo por situações fora do controle da instituição:

a) Neste caso, entende-se que cada discente faltou a avaliação, então será atribuída nota zero a todos os critérios de avaliação;
b) Neste caso, cada discente deverá solicitar uma segunda chamada com os documentos comprobatórios à coordenadoria do curso conforme as orientações do Regulamento de Organização Didática (ROD[1]) do IFCE;

[1] Disponível em https://ifce.edu.br/espaco-estudante/regulamento-de-ordem-didatica

X. Após a correção desta atividade, em caso de discordância da nota E negativa do professor em atender suas solicitações, é preciso seguir as orientações do artigo 96 do ROD do IFCE.

XI. Dúvidas surgidas na aplicação desta avaliação, bem como os casos omissos, serão resolvidas pelo docente da disciplina via e-mail adonias.oliveira@ifce.edu.br ;

XII. Em casos extremos não previstos, será consultada a Coordenadoria Técnico-Pedagógica (CTP), bem como, será relatado qualquer situação inadequada à Coordenadoria do Curso de Ciência da Computação do IFCE Campus Tianguá.

QUESTÕES GERAIS (TODAS EQUIPES)

A) Construa um analisador léxico usando Python e PLY que leia uma equação do 1º grau ($ax + b = 0$) e reconheça os tokens que representam os números, variáveis, operadores (+, -, *, /) e o operador de igualdade, conforme é exemplificado abaixo. Deve ser implementada uma interface de dois campos: entrada da equação do 1º grau e saída da análise léxica. Um botão chamado "Analisar lexicamente" deve ser implementado. Para a interface é sugerido a API Gradio[2].

5 * x + 10 = 0

TOKEN	TIPO	VALOR
5	NÚMERO	5
*	OPERADOR	MULTIPLICAÇÃO
x	VARIÁVEL	x
+	OPERADOR	SOMA
10	NÚMERO	10
=	IGUALDADE	IGUALDADE
0	NÚMERO	0

B) Construa um compilador básico em Python (analisador léxico + sintático + gerador de código) que leia um arquivo txt com expressões aritméticas reconhecendo os tokens que representam os números, parênteses e operadores (+, -, *, /), conforme é exemplificado abaixo.

5 * (2 + 10)

TOKEN	TIPO	VALOR
5	NÚMERO	5
*	OPERADOR	MULTIPLICAÇÃO

[2] Disponível em https://gradio.app/guides/

(	PARÊNTESE ABERTO	(
2	NÚMERO	2
+	OPERADOR	SOMA
10	NÚMERO	10
)	PARÊNTESE FECHADO	)

Obs.: Outras expressões aritméticas serão testadas no seu código.

EQUIPE 01

1) Implemente um programa de ordenação Bolha utilizando a linguagem do software VERTO. Entenda-se programa o código com a função principal. Após compilar o código pelo simulador VERTO, exiba a tabela de símbolos, o código macroassembler e as saídas do analisador léxico, sintático e semântico. Faça um comentário sobre as análises realizadas. Houve sucesso ou fracasso na compilação? Justifique.

2) Classifique a gramática abaixo como Linear à Direita ou à Esquerda e se são unitárias ou não. Justifique sua resposta.

$$
\begin{aligned}
S_1 &\rightarrow abcdP \\
P &\rightarrow efP \\
P &\rightarrow Q \\
Q &\rightarrow g
\end{aligned}
$$

3) Utilizando JFlap, contrua um autômato de pilha não determinístico cuja linguagem aceita é $L = \{a^n b^m \in \{0,1\}^* \mid m;\ n \geq 0\ e\ m \neq n\}$;

4) Simule **Linguagem formado por parênteses** na ferramenta JFlap e exiba os resultados (se a cadeia é aceita e a árvore de derivação).

Especificação das regras de produção:

S → SS
S → (S)
S → ()
S → λ

Sentenças:

- ()() - Válida
- ())() - Inválida
- (())() – Válida

5) Construa no GALS um analisador sintático para expressões aritméticas com soma e multiplicação, respeitando precedência.

Gramática sugerida:

E → E + T | T

T → T * F | F

F → (E) | id

a) Inserir a gramática no GALS

b) Testar as expressões:

- id + id * id
- (id + id) * id
- id *

c) Verificar se a precedência foi respeitada na análise

6) Modifique o código da Aula 07 (Análise Léxica em C) para que o analisador léxico reconheça também o operador de módulo (%) e o operador de potenciação (^). Os novos operadores deverão ser identificados e exibidos, respectivamente, como MOD e POT. Exemplo de entrada: 10 % 3 + 2 ^ 4

7) **Reconhecimento de expressões aritméticas.** Utilizando Java e JFlex, implemente um analisador léxico capaz de reconhecer os tokens presentes em expressões aritméticas. O analisador deverá reconhecer:

- números inteiros;
- operadores +, -, *, / e ^;
- parênteses (e);
- espaços em branco, que deverão ser ignorados.

Teste com as expressões:

- 10 + 20 * 3
- (8 - 2) / 3
- 2 ^ 4 + 5

Para cada expressão, apresente a sequência de tokens identificados.

EQUIPE 02

1) Implemente um programa de ordenação SELEÇÃO (Selection Sort) utilizando a linguagem do software VERTO. Entenda-se programa o código com a função principal. Após compilar o código pelo simulador VERTO, exiba a tabela de símbolos, o código macroassembler e as saídas do analisador léxico, sintático e semântico. Faça um comentário sobre as análises realizadas. Houve sucesso ou fracasso na compilação? Justifique.

2) Classifique a gramática abaixo como Linear à Direita ou à Esquerda e se são unitárias ou não. Justifique sua resposta.

$$S \rightarrow Aab$$
$$A \rightarrow Aab \mid B$$
$$B \rightarrow a$$

3) Utilizando JFlap, contrua um autômato de pilha não determinístico cuja linguagem aceita é $L = \{w \in \{0,1\}^* \mid w = w^r$, ou seja, w é palíndromo$\}$

4) Simule **Linguagem formado por parênteses e colchetes** na ferramenta JFlap e exiba os resultados (se a cadeia é aceita e a árvore de derivação).

Especificação das regras de produção:

S → SS
S → ()
S → (S)
S → []
S → [S]
S → λ

Sentenças:

- () - Válida
- [] - Válida
- [()()] - Válida
- [([()])] - Válida
- [(]) - Inválida
- [(([[]]))] – Válida

5) Considere a linguagem de parênteses balanceados. No GALS, implemente uma gramática que reconheça essa linguagem e simule a análise sintática para as cadeias fornecidas. Gramática sugerida:

$S \rightarrow (S)S \mid \varepsilon$

a) Implemente a gramática no GALS

b) Teste as cadeias abaixo e informe se são aceitas ou rejeitadas:

- (())
- ()()
- (()
- ())(

c) Observe o comportamento da pilha (se usar modo LL/LR)

6) Modifique o código da Aula 07 (Análise Léxica em C) para que o analisador reconheça números reais, além dos números inteiros já suportados. O caractere . deverá fazer parte do token numérico quando estiver entre dígitos. Exemplo de entrada: 10.5 + 2.75 * 3. O analisador deverá identificar 10.5 e 2.75 como tokens do tipo número.

7) **Reconhecimento de identificadores, números e atribuição.** Utilizando Java e JFlex, implemente um analisador léxico para reconhecer comandos simples de atribuição. O analisador deverá reconhecer:

- identificadores, como x, valor e nota_final;
- números inteiros e reais;
- operador de atribuição =;
- operadores +, -, * e /;
- ponto e vírgula ;;
- espaços em branco, que deverão ser ignorados.

Teste com as entradas:

- x = 10;
- media = 7.5;
- resultado = valor1 + valor2 * 2;

Informe os tokens reconhecidos em cada entrada.

EQUIPE 03

1) Implemente o método de ordenação INSERÇÃO (Insertion Sort) utilizando a linguagem do software VERTO. Entenda-se programa o código com a função principal. Após compilar o código pelo simulador VERTO, exiba a tabela de símbolos, o código macroassembler e as saídas do analisador léxico, sintático e semântico. Faça um comentário sobre as análises realizadas. Houve sucesso ou fracasso na compilação? Justifique.

2) Classifique a gramática abaixo como Linear à Direita ou à Esquerda e se são unitárias ou não. Justifique sua resposta.

$$
\begin{aligned}
S_2 &\rightarrow aP_1 \\
P_1 &\rightarrow bP_2 \\
P_2 &\rightarrow cP_3 \\
P_3 &\rightarrow dP \\
P &\rightarrow eP_4 \\
P_4 &\rightarrow fP \\
P &\rightarrow Q \\
Q &\rightarrow g
\end{aligned}
$$

3) Utilizando JFlap, contrua um autômato de pilha não determinístico cuja linguagem aceita é:

$L = \{w \in \{0,1\}^* \mid o\ comprimento\ de\ w$ é $impar\ e\ o\ símbolo\ do\ meio$ é $zero\}$;

4) Simule Linguagem formada por pares de letras na ferramenta JFlap e exiba os resultados (se a cadeia é aceita e a árvore de derivação).

Especificação das regras de produção:

S → aSb
S → ab
S → λ

Sentenças:

- ab -Válida
- abab - Inválida
- aaabbb – Valida

5) Considere a linguagem formada por cadeias com o mesmo número de a's seguidos de b's, ou seja, a linguagem $a^n b^n$. Modele essa linguagem no GALS.

Gramática sugerida:

$S \rightarrow aSb \mid \varepsilon$

a) Implementar a gramática no GALS

b) Testar as cadeias:

- ab
- aabb
- aaabbb
- aab

c) Explicar por que a cadeia inválida é rejeitada

6) Modifique o código da Aula 07 (Análise Léxica em C) para que sejam reconhecidos identificadores formados por letras, permitindo analisar expressões que contenham variáveis. Crie um novo tipo de token denominado TOK_ID. Exemplo de entrada: valor + total * 10. O analisador deverá reconhecer valor e total como identificadores, 10 como número e + e * como operadores.

7) **Reconhecimento de operadores relacionais e lógicos.** Utilizando Java e JFlex, implemente um analisador léxico capaz de reconhecer expressões relacionais e lógicas. O analisador deverá reconhecer:

- identificadores;
- números inteiros;
- operadores relacionais >, <, >=, <=, == e !=;
- operadores lógicos &&, || e !;
- parênteses.

Teste com as expressões:

- idade >= 18
- nota >= 7 && faltas < 10
- !(x == 10) || y != 5

Apresente a sequência de tokens gerada pelo JFlex para cada expressão.

EQUIPE 04

1) Implemente um programa de ordenação Heap Sort utilizando a linguagem do software VERTO. Entenda-se programa o código com a função principal. Após compilar o código pelo simulador VERTO, exiba a tabela de símbolos, o código macroassembler e as saídas do analisador léxico, sintático e semântico. Faça um comentário sobre as análises realizadas. Houve sucesso ou fracasso na compilação? Justifique.

2) Classifique a gramática abaixo como Linear à Direita ou à Esquerda e se são unitárias ou não. Justifique sua resposta.

$$\begin{aligned} S &\rightarrow aA \mid bB \mid a \\ A &\rightarrow aA \mid bB \mid B \\ B &\rightarrow bB \mid a \end{aligned}$$

3) Utilizando o JFLAP, construa um autômato a pilha não determinístico cuja linguagem aceita é: $L = \{0^n 1^n \mid n \geq 0\}$, ou seja, a linguagem formada por cadeias que possuem uma quantidade de símbolos 0 igual à quantidade de símbolos 1, sendo que todos os 0s devem aparecer antes dos 1s.

Exemplos de cadeias aceitas: (ε),01,0011,000111.
Exemplos de cadeias rejeitadas: 0, 1, 001, 011, 0101, 00011.

4) Simule a linguagem formada por palíndromos sobre o alfabeto ($\{a, b\}$) na ferramenta JFLAP e exiba os resultados, indicando se cada cadeia é aceita ou rejeitada e apresentando a árvore de derivação para as cadeias aceitas.

Especificação das regras de produção:

$$\begin{aligned} S &\rightarrow aSa \\ S &\rightarrow bSb \\ S &\rightarrow a \\ S &\rightarrow b \\ S &\rightarrow \lambda \end{aligned}$$

Sentenças:
aba – Válida
abba – Válida
aabbaa – Válida
abab – Inválida
baab – Válida
abbba – Válida

5) Considere a linguagem formada por palíndromos sobre o alfabeto ($\{a, b\}$), ou seja, cadeias que possuem a mesma sequência quando lidas da esquerda para a direita ou da direita para a esquerda. Modele essa linguagem no GALS.

Gramática sugerida:

$$S \rightarrow aSa \mid bSb \mid a \mid b \mid \varepsilon$$

a) Implemente a gramática no GALS.
b) Teste as cadeias abaixo e informe se são aceitas ou rejeitadas:

aba – Válida
abba – Válida
aabbaa – Válida
abab – Inválida

c) Para uma das cadeias válidas, apresente a sequência de derivações realizada a partir do símbolo inicial (S) até a obtenção da cadeia.

d) Explique por que a cadeia abab não pode ser gerada pelas regras de produção da gramática.

6) Modifique o código da Aula 07 (Análise Léxica em C) para que o analisador seja capaz de identificar e informar caracteres inválidos, em vez de simplesmente encerrar a análise ao encontrar um caractere não reconhecido. analisador_lexico. Exemplo de entrada: 10 + 5 @ 2. O analisador deverá reconhecer normalmente os tokens válidos e apresentar uma mensagem como Erro léxico: caractere '@' não reconhecido, indicando também a posição do caractere na expressão.

7) **Reconhecimento de palavras reservadas de uma linguagem.** Utilizando Java e JFlex, implemente um analisador léxico simplificado para uma linguagem de programação. O analisador deverá diferenciar palavras reservadas de identificadores. Considere as seguintes palavras reservadas:

if, else, while, int, float, return

Além delas, o analisador deverá reconhecer identificadores, números inteiros, operadores aritméticos, operadores relacionais, =, ;, {, } e parênteses.

Teste o analisador com o seguinte trecho:

```
int idade = 20;

if (idade >= 18) {
   return idade;
}
```

a) Implemente as expressões regulares e regras correspondentes no arquivo .flex.
b) Gere o analisador léxico utilizando o JFlex.
c) Execute o analisador e apresente a sequência de tokens reconhecidos.
d) Explique como o analisador diferencia a palavra reservada int de um identificador como idade.